SYSTEM DESIGN TRAINING · DAY 4

Database Optimization, Sharding & Microservices Architecture

Two levers on the database we already have, then the bigger move: splitting the application itself

Case study: RetailOrderHub — one database, then one application, both put under real pressure

Today's Agenda

Two earlier levers on the same database, then the bigger move: splitting the application itself

01	**Database Optimization**	Indexing, replication & failover, polyglot persistence — plus a hands-on indexing lab
02	**Database Sharding**	What sharding is, sharding strategies, hashing & rehashing challenges
03	**Microservices & Data Ownership**	Why split the application — plus a hands-on decomposition lab
04	**Break**	Stretch, refill, reset
05	**Database-per-Service vs. Shared Database**	Both approaches, benefits & drawbacks, cross-service queries, CAP theorem
06	**Distributed Transactions & the Saga Pattern**	Why transactions break across services — Saga flow, choreography vs. orchestration
07	**CQRS Pattern**	Command/query separation — RetailOrderHub's read and write sides, and when to use it

RUNNING CASE STUDY

RetailOrderHub, Going Into Day 4

Two separate questions today, not one: how far we can scale the database we already have, and — later — how the application itself is split into services.

WHERE WE LEFT OFF (DAY 3)

One RetailOrderHub Application
+ One Relational Database

- Order, Customer, Inventory, and Payment data all live in one database
- Order and Customer tables are the largest and fastest-growing
- Slow queries are starting to show up on the Orders dashboard

TODAY, IN ORDER

1. Optimize the single database — index it, then shard it

2. Split the application — Order, Inventory, Payment as services

The database question comes first because it's the lever we already know. The application question is the bigger, newer move — and it's the one that creates everything from CAP theorem to the Saga pattern later today.

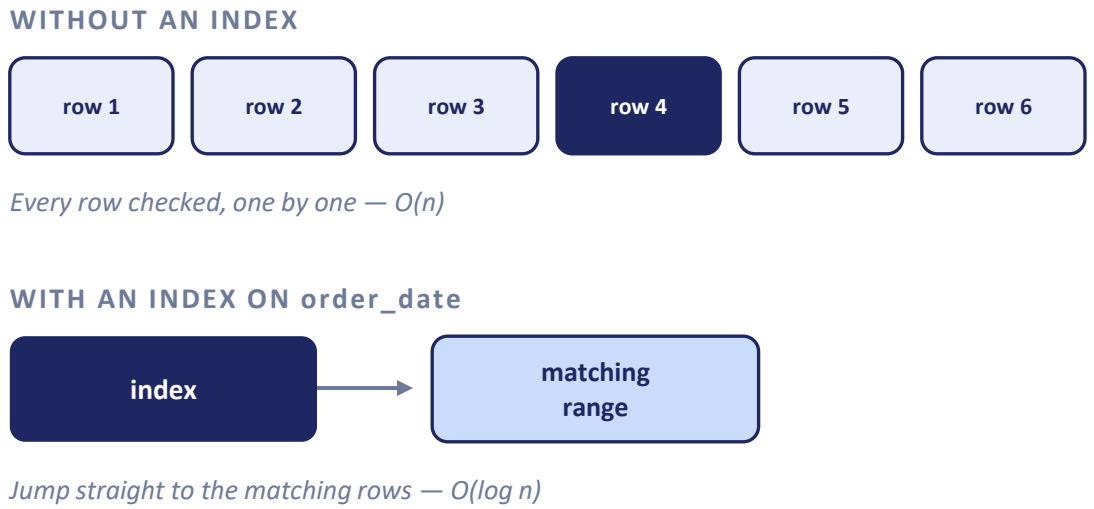
BLOCK 1

Database Optimization

Getting more out of the database we already have, before we reach for anything bigger.

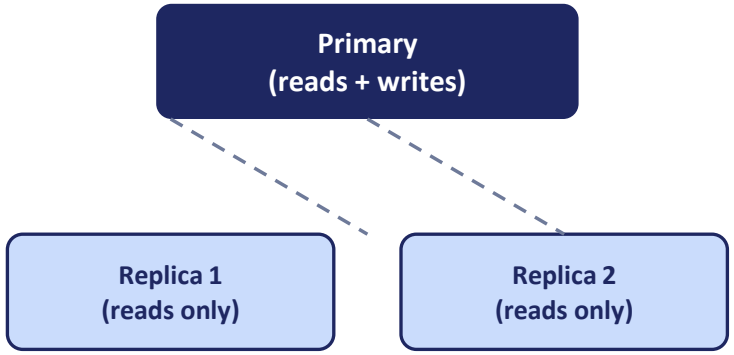
Indexing: Improving Query Performance

- An index is a sorted lookup structure the database keeps alongside a table
- Without one, every query scans every row — a table scan, and it gets slower as the table grows
- Index the columns used in WHERE, JOIN, and ORDER BY clauses — that's where RetailOrderHub's Orders dashboard is slow today
- Trade-off: faster reads, but extra storage and slower writes for every index added
- Composite indexes should match the query's filter and sort order, not just cover one column



Replication and Failover

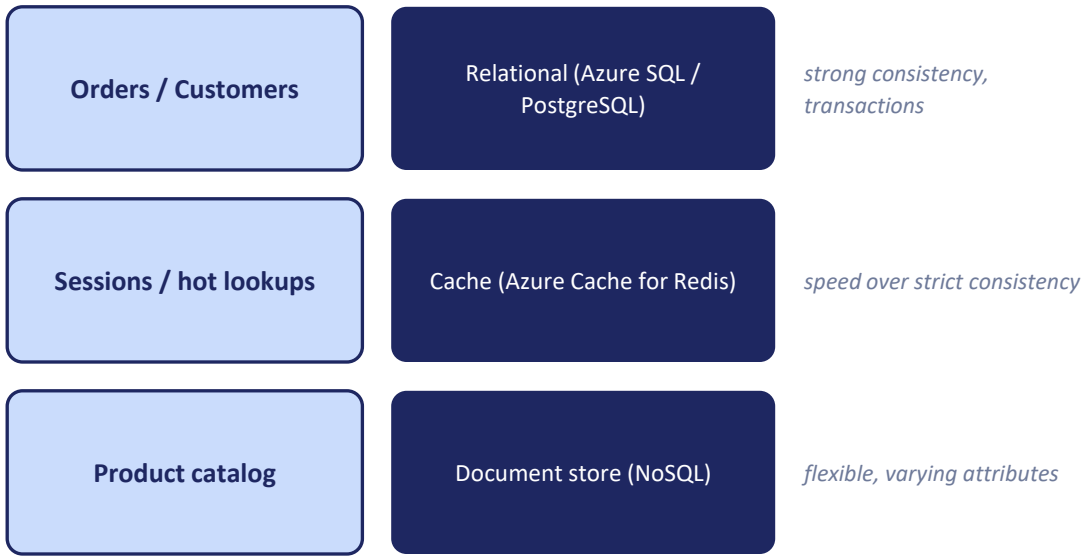
- Replication keeps continuously synced copies of the database on other nodes
- The primary handles writes; one or more replicas can serve reads
- If the primary fails, failover promotes a replica to primary automatically
- Improves both read throughput and disaster recovery
- Watch for replication lag — a replica can briefly serve stale reads right after a write



If Primary fails — a replica is promoted automatically

Polyglot Persistence

- Use the database engine that fits each kind of data, not one engine for everything
- Orders and Customers: a relational store — strong consistency and transactions
- Session data and hot product lookups: a cache — speed over strict consistency
- A product catalog with varying attributes: a document store — flexible schema
- Trade-off: more moving parts and more operational overhead to manage



HANDS-ON LAB

Add and Tune Indexes on RetailOrderHub

Work against the Order and Customer tables in your Azure SQL / PostgreSQL environment.

1. Run the provided baseline queries against the Order and Customer tables and capture the execution plan.
2. Identify which columns are driving each WHERE, JOIN, or ORDER BY clause.
3. Add a targeted index — single-column or composite — for the slowest query.
4. Re-run the same query and compare the before/after execution plan and duration.
5. Discuss: which index helped most, and what did it cost on the write side?

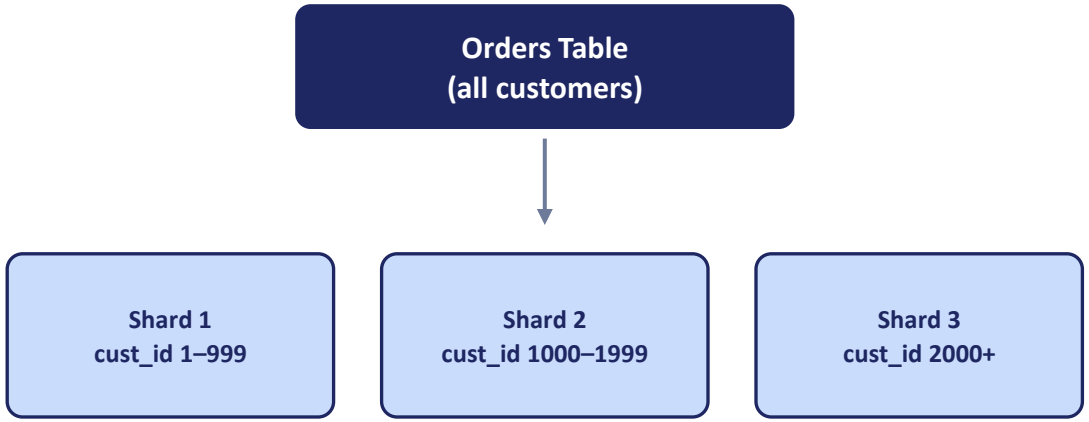
BLOCK 2

Database Sharding

When one machine — even indexed and replicated — isn't enough, we split the data across many.

What Is Sharding?

- Sharding splits one large table's rows across several independent databases
- Each shard holds a subset of rows — together, they hold the full table
- A shard key (e.g. customer_id) decides which shard a given row lives on
- Different from replication: shards hold different rows; replicas hold copies of the same rows
- Different from vertical partitioning: sharding splits rows, not columns, across databases



Applied to RetailOrderHub: each shard holds a slice of customers' orders — together they cover the whole dataset.

Sharding Strategies

Range-based

Split by a value range, like order_date or customer_id ranges

Hash-based

Apply a hash function to the shard key to pick the shard

Directory-based

A lookup service maps each key to its shard explicitly

For RetailOrderHub: shard Orders by hashed customer_id, so each customer's order history lands on one shard.

Hashing, Rehashing & Sharding Challenges

- hash(shard_key) mod N picks the shard for a given key
- Rehashing problem: adding or removing shards changes N, so most keys remap — that's expensive
- Consistent hashing minimizes how many keys move when N changes
- Cross-shard queries and joins: no single shard has the full picture
- Rebalancing and hotspots: uneven key distribution can overload one shard

BLOCK 3

Microservices & Data Ownership

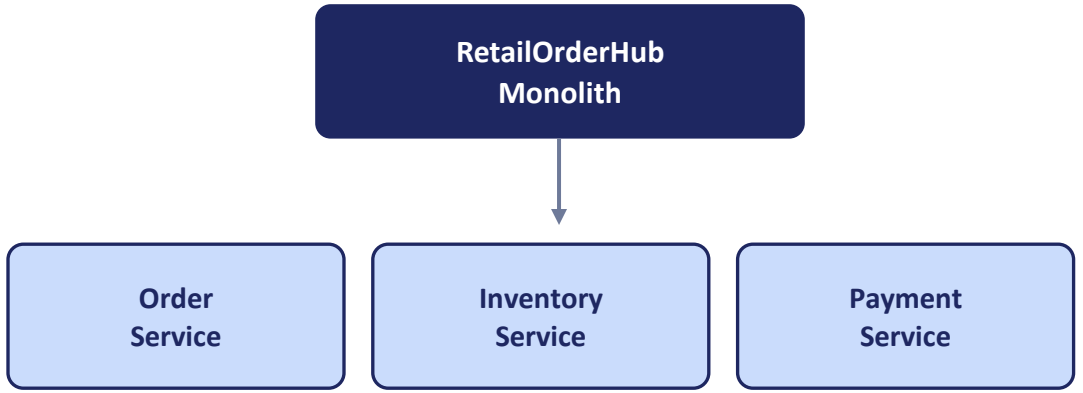
We've scaled the database as far as indexing and sharding take it. Now a different lever: splitting the application itself.

Why Break Up the Monolith?

- One codebase means one deploy — a one-line Payment fix waits behind whatever else is mid-flight in Order or Inventory
- One process means one blast radius — a bug in the Inventory code path can take down Order and Payment too
- One deployable means one scaling knob — a spike in Order traffic scales Payment and Inventory right along with it
- Sharding the database, on its own, doesn't fix any of this — it's still one application, one deploy, one blast radius
- Microservices trade this for independence — at the cost of the network, consistency, and operational complexity we'll spend the rest of today on

RetailOrderHub, Decomposed

- Split along business capability — Order, Inventory, Payment — not technical layers like “web tier” or “data tier”
- Each service owns its own logic end to end, and exposes it only through an API
- Data ownership rule: only the owning service ever touches its own data — everyone else asks through that API
- No other service reaches into Order's data, Inventory's data, or Payment's data directly — not even for a quick read
- This rule is what every pattern for the rest of today exists to protect



No arrows drawn between the three boxes on purpose — direct service-to-service data access is exactly what the ownership rule forbids.

HANDS-ON LAB · GROUP EXERCISE

Decompose the RetailOrderHub Monolith

In groups, review the current RetailOrderHub monolith's modules and data model.

1. Propose service boundaries: OrderService, InventoryService, PaymentService.
2. For each service, define what data it owns, and what it must not reach into directly.
3. Identify the touchpoints where services will need to call one another.
4. Present your boundaries — and be ready to defend a debatable call.

Break

Grab a coffee — we're back to weigh database-per-service against a shared database.

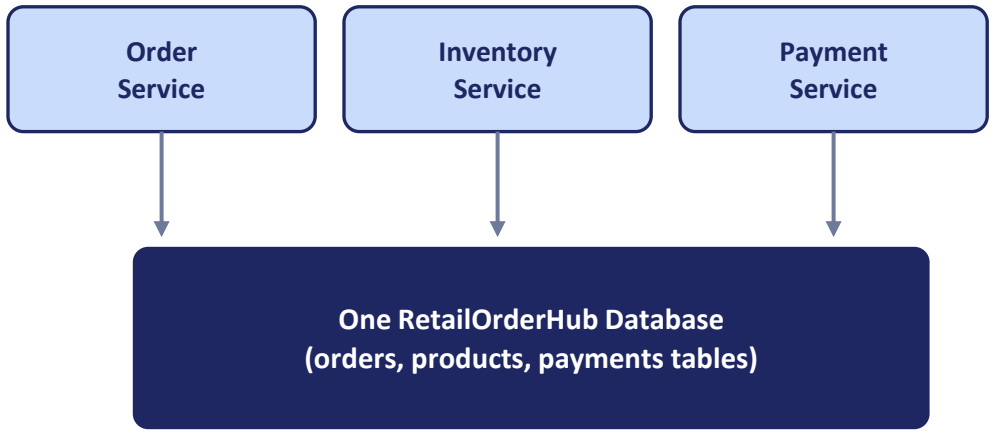
BLOCK 4

Database-per-Service vs. Shared Database

The data-ownership question we set aside in Block 3 — answered now, with both sides on the table.

Option A: The Shared Database

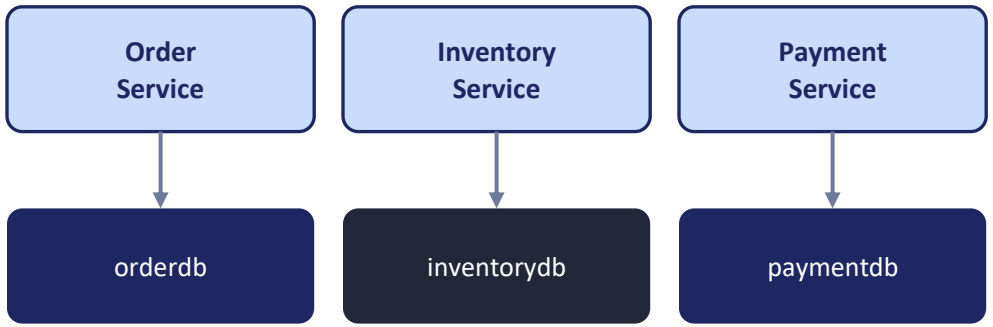
- All three services — Order, Inventory, Payment — read and write the same physical database
- A service boundary in code, but one shared schema underneath it
- Any service can JOIN across Orders, Products, and Payments directly — no API call needed
- One transaction can span all three tables with real ACID guarantees — the database enforces it
- The trade-off: the services aren't actually independent — a schema change for Payment can silently break Order's queries



Every service can see — and query — every table.

Option B: Database-per-Service

- Each service gets its own database — Order owns orderdb, Inventory owns inventorydb, Payment owns paymentdb (or none, if stateless)
- No other service can query another's database directly — not even read-only
- The only way to reach another service's data is through its API
- Each service can even pick its own storage engine — relational for Orders, a cache for hot Inventory lookups
- The trade-off: cross-service JOINs and cross-service transactions are both gone — that's the subject of the rest of today



No lines drawn between the three databases — each is reachable only through its own service's API.

Comparing the Two: Benefits & Drawbacks

Shared Database

BENEFITS

- ✓ Real cross-table JOINs, no extra hop
- ✓ One database transaction can span Order, Inventory, Payment with true ACID guarantees
- ✓ Simple mental model — one place to look for data

DRAWBACKS

- ✗ Services aren't really independent — one schema change can break another service's queries
- ✗ One database is now everyone's scaling and availability bottleneck
- ✗ No freedom to pick a different storage engine per service

Database-per-Service

BENEFITS

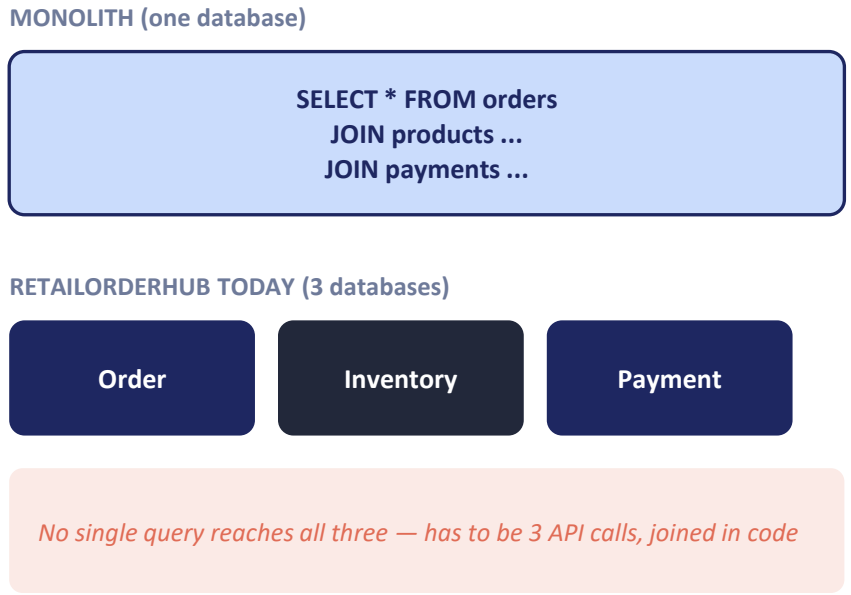
- ✓ Services deploy and scale truly independently — no shared schema to coordinate
- ✓ Each service can choose the storage engine that fits its data
- ✓ A failure or slowdown in one database doesn't directly touch another service's data

DRAWBACKS

- ✗ No cross-service JOINs — composing data means calling multiple APIs
- ✗ No cross-service ACID transactions — needs patterns like Saga (next block)
- ✗ More infrastructure to run and operate — N databases instead of one

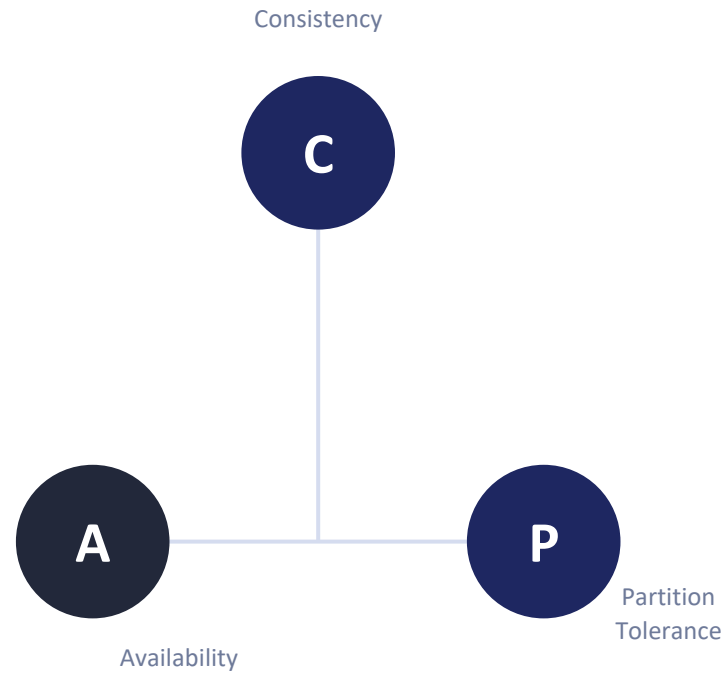
The New Problem: Cross-Service Queries

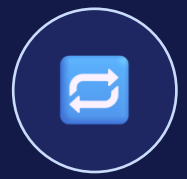
- “Show me this customer's order history with product names and payment status” used to be one SQL JOIN across three tables
- In RetailOrderHub today, that data lives in three separate databases behind three separate APIs
- One option: call all three services, then stitch the results together in application code — slower, and three points of failure instead of one query
- This is exactly the cost we accepted two slides ago by choosing database-per-service
- Later today, CQRS gives this specific problem a real answer — keep that question in mind



CAP Theorem: A Tool for This Exact Decision

- Once each service has its own database, that database is a distributed system — CAP theorem is how we reason about it
- Consistency — every read gets the most recent write, everywhere
- Availability — every request gets a non-error response
- Partition Tolerance — the system keeps working even when a network split happens between nodes
- Real systems can't guarantee all three during a partition — every distributed database has already chosen CP or AP, whether its team decided that on purpose or not
- Discuss: RetailOrderHub's Inventory Service, split across two data centers — during a partition, would you rather risk overselling (AP) or reject valid orders (CP)?





BLOCK 5

Distributed Transactions & the Saga Pattern

We gave up cross-service ACID transactions on purpose. Here's how we get consistency back.

Why Distributed Transactions Break Across Services

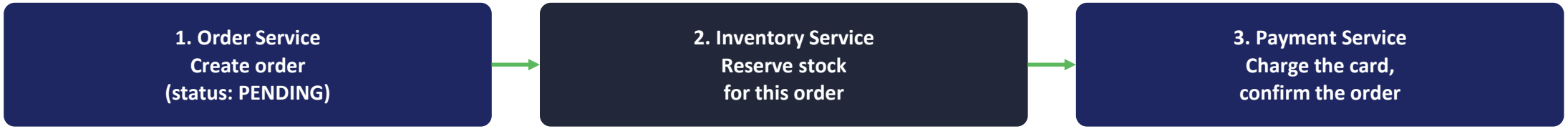
- Placing a RetailOrderHub order touches three services: reserve stock (Inventory), charge the card (Payment), record the order (Order)
- In the monolith, this was one database transaction — all three steps committed together, or none did
- Now it's three separate calls to three separate databases — there's no single COMMIT that covers all three
- If step 2 fails after step 1 already succeeded, nothing automatically undoes step 1
- Two-phase commit (2PC) can technically coordinate this, but it locks every database until every service responds — almost nobody runs it across independent microservices

What Is a Saga?

- A saga is a sequence of local transactions, one per service, instead of one distributed transaction
- Each local transaction commits on its own, in its own database, immediately
- Each step also defines a compensating transaction — an action that undoes it, if a later step fails
- If step 3 fails, the saga runs the compensating transactions for steps 2 and 1, in reverse order
- This is not the same as rollback — the earlier steps already committed and are visible; a compensation is a new transaction that cancels their effect

RetailOrderHub's Saga, Step by Step

HAPPY PATH



IF THE CHARGE FAILS — COMPENSATION RUNS IN REVERSE

Compensation runs right-to-left — undo the most recent successful step first, then the one before it.

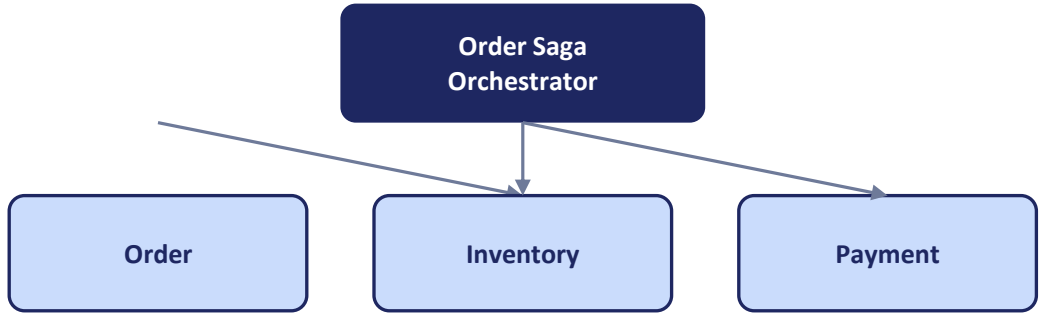
Who decides to run that compensation, and in what order? That's a real design choice — services can figure it out among themselves (choreography), or one coordinator can direct every step (orchestration). Next slide.

Choreography vs. Orchestration

CHOREOGRAPHY — services react to each other's events

- No central coordinator — each service publishes an event, the next one listens and reacts
- Loosely coupled, easy to add a new participant
- Hard to see the whole flow in one place; tracing a failure means following events across every service's logs

ORCHESTRATION — one coordinator directs every step



- One orchestrator calls each service directly and tracks saga state
- The whole flow — and the compensation logic — lives in one place, easy to read and test
- That orchestrator is now a central point every step depends on — more coupling, one more thing to keep highly available

BLOCK 6

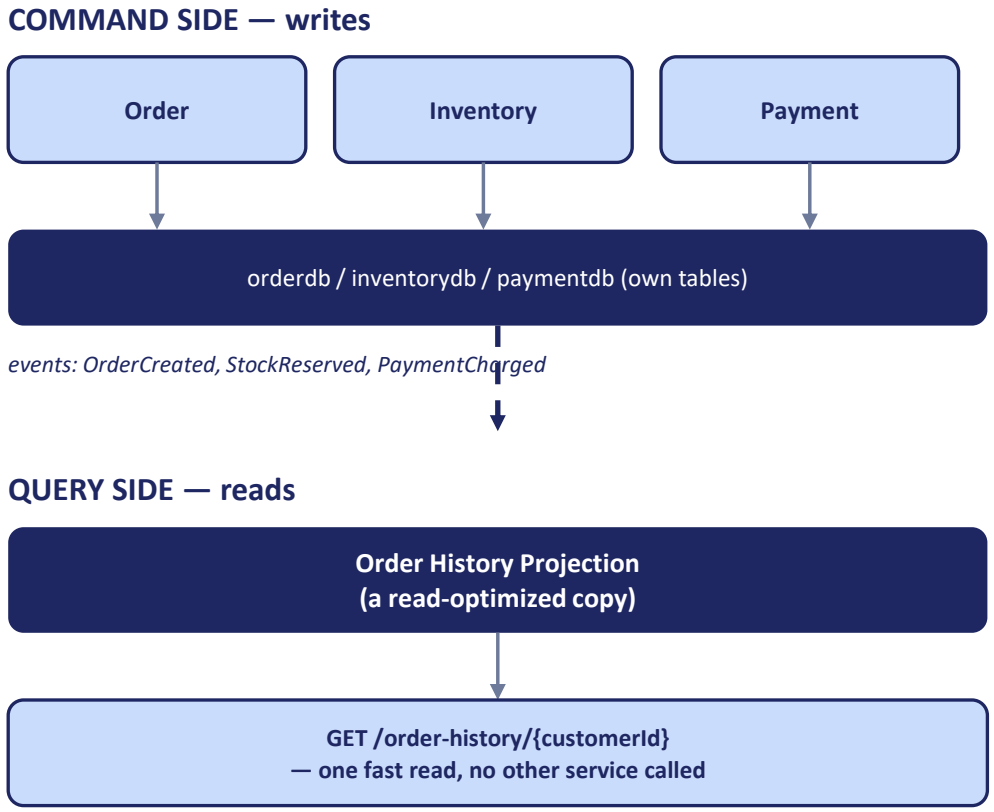
CQRS Pattern

A real answer to the cross-service query problem we hit in Block 4.

What Is CQRS?

- CQRS = Command Query Responsibility Segregation — use a different model for writing data than for reading it
- Commands (writes) go through each service's normal API — PlaceOrder, ReserveStock, ChargeCard — exactly as we've built all day
- Queries (reads) go against a separate read model, built specifically to answer one question fast, with no cross-service calls at request time
- That read model is kept up to date by listening to the same events each service already publishes for the saga
- The trade-off: the read model is eventually consistent — it can lag the write side by a moment, in exchange for reads that are simple and fast

RetailOrderHub's Command Side and Query Side



- The command side never changed — each service still owns its writes exactly as before
- The projection is a denormalized copy — order + product name + payment status already joined together, built for this one screen
- If Inventory renames a product, that shows up in the projection a moment later, once the event arrives — not instantly
- This is the direct fix for the cross-service query problem from Block 4 — one read, not three

When to Use CQRS — and When Not To

- Use it where read and write patterns genuinely diverge — RetailOrderHub's order history is read constantly, by many different views, and written once
- Don't apply it everywhere by default — it adds a whole extra data pipeline that has to be built and operated
- Every CQRS read model is eventually consistent — if a screen needs the absolute latest write visible immediately, CQRS isn't the right tool for that screen
- A good sign you need it: your team keeps reaching for cross-service calls just to render one page, and that page is read far more often than the data changes
- A good sign you don't: a single service's own straightforward CRUD screen — that's just a normal query against that service's own database

DAY 4 RECAP

Two Levers on the Database, One Big Move on the Application

- Optimized RetailOrderHub's single database — indexing, replication and failover, polyglot persistence.
- Sharded that same database — strategies, hashing and rehashing, and the challenges that come with it.
- Split the application itself into services, and drew a hard line: each service owns its own data.
- Weighed Database-per-Service against a Shared Database — real benefits and drawbacks on both sides — and felt the cross-service query problem it creates.
- Used CAP theorem to reason about CP vs. AP for a genuinely distributed database.
- Replaced the cross-service transaction we gave up with the Saga pattern, and answered the cross-service query problem with CQRS.

Tomorrow: how services actually talk to each other at scale, service mesh, and Istio.